SESSION 13: File Handling & Database Connectivity

SESSION OVERVIEW:

In this session, the students will be able to:

- Understand how to read from files
- Understand the importance and methods of closing files in Python
- Understand how to write data to a file
- Understand how to create a new file
- Understand how to delete a file
- Understand the purpose and use of libraries in Python
- Understand the Importance of Database Connectivity
- Use Python Libraries for Database Connectivity
- Discuss Database Connection Parameters
- Executing SQL Queries

KEY POINTS AND EXAMPLES:

Understand how to read from files

To read from a file in Python, you first need to open it using the open() function. Specify the file path and the mode 'r' for reading.

```
file = open("file.txt", "r")
```

Let's assume we have a file named file.txt with the following content:

```
This is a sample file.
It contains some text.
Good Luck!
```

Once the file is opened, you can use the following methods to read its contents:

1. **read() Method**

   The read() method reads the entire contents of the file and returns it as a single string. If you don't specify any arguments, it reads the entire file. However, you can also specify the number of bytes to read.

```
file = open("file.txt", "r")
# Read the entire file
content = file.read()
print("Entire file content:")
print(content)
```

- Reading the Entire File Content: file.read() reads the entire content of the file file.txt into the variable content.
- Printing the Entire File Content: print(content) displays the entire content of the file file.txt as a single string.

The output will be:

```
Entire file content:
This is a sample file.
It contains some text.
Good Luck!
```

The default mode for opening a file in Python is 'r', which stands for read mode. When you use the open() function without specifying a mode, it opens the file for reading by default.

Note: If you try to open a non-existent file in read mode ('r'), Python raises a FileNotFoundError. This error indicates that the file you are trying to read does not exist in the specified location.

Read Only Parts of the File: By default the read() method returns the whole text, but you can also specify how many characters you want to return:

```
file = open("file.txt", "r")
# Return the 5 first characters of the file
print(file.read(5))
```

The output will be:

```
This
```

2. **readline() Method**

   The readline() method reads one line from the file.It stops reading each line when it reaches the end of the line or finds a newline character. The readline() method in

Python does not consider punctuation marks like full stops (.), commas (,), or any other punctuation as special characters that affect its behavior.

```
file = open("file.txt", "r")
# Read the first line
line1 = file.readline()
print("First line:", line1)

# Read the next line
line2 = file.readline()
print("Next line:", line2)
```

The output will be:

```
First line: This is a sample file.
Next line: It contains some text.
```

After the line "This is a sample file.", there is a newline character (\n).
After the line "It contains some text.", there is another newline character (\n).
The line "Good Luck!" is the last line and does not have a newline character (\n).

3. **readlines() Method**

   The readlines() method reads all lines of the file into a list of strings. Each element in the list corresponds to a line from the file, including the newline character (\n) at the end of each line.

```
file = open("file.txt", "r")
# Read all lines into a list
line = file.readlines()
print(line, end="")
```

Reading All Lines into a List: file.readlines() reads all lines from the file file.txt and stores them as strings in a list named line. Each element of the list corresponds to a line from the file, including newline characters (\n).

The output will be:

```
['This is a sample file.\n', 'It contains some text.\n', 'Good
Luck!\n']
```

Looping over File Contents: We can easily loop over the contents of a file using a for loop.

```
file = open("file.txt", "r")
for line in file:
    print(line, end="")
```

The output will be:

```
This is a sample file.
It contains some text.
Good Luck!
```

Understand the difference between read(), readline(), and readlines()

Method	Description	Example	Output Example	When to use it:
read()	Reads the entire file or a specified number of characters..	content = file.read()	Reads all content as a single string.	When you need to process the entire content of the file at once.
readline()	Reads one line at a time until it encounters a newline (\n).	line = file.readline()	Reads the next line including the newline character.	When you want to process the file line by line, especially useful for large files.
readlines()	Reads all lines and returns them as a list of strings.	lines = file.readlines()	Reads all lines and returns a list where each element is a line.	When you want to read all lines at once but process them individually as elements of a list.

Understand the importance and methods of closing files in Python

Closing files in Python is crucial to ensure efficient resource management and prevent potential issues related to file handling.

When a file is opened for reading or writing in Python, it's important to close it properly once you're done with it.

Importance of Closing Files:

- Release of Resources: Closing files releases system resources (like memory) that were allocated when the file was opened. Failing to close files can lead to resource leaks, especially when dealing with a large number of files or in long-running programs.
- Data Integrity: Closing files ensures that all data has been written to the file and that any internal buffers are flushed. If you don't close a file properly, some data may not be saved.
- Concurrency: Closing files properly can prevent issues related to file access concurrency, especially when multiple processes or threads are accessing the same file.

Methods to Close Files:

1. **Using close() Method:** Call the close() method on the file object to close it explicitly. This is the most direct way to close a file.

Syntax:

```
file = open('file.txt', 'r')
# File operations...
file.close()  # Closing the file explicitly
```

In this approach, after performing operations like reading or writing, you must remember to explicitly close the file by calling file.close().

2. **Using try...finally Block:** Another method is to use a try...finally block to ensure that the file is closed even if an exception occurs during file operations. This approach guarantees that the file will be closed regardless of whether an exception is raised.

```
file = open('file.txt', 'r')
try:
    # File operations...
    pass
finally:
    file.close()  # Closing the file in the finally block
```

Here, the finally block ensures that file.close() is always called, even if an exception occurs in the try block.

```
try:
    # Open the file in read mode ('r' for read)
    file = open("file.txt", 'r')

    try:
```

```
        # Perform file operations
        content = file.read()
        print("File content:")
        print(content)

    except IOError:
        print("Error: An IOError occurred. Details")
    except Exception:
        print("Error: An unexpected error occurred. Details")

    finally:
        # Close the file
        file.close()

except FileNotFoundError:
    print("Error: The file does not exist or cannot be found.")
except Exception:
    print("Error: An unexpected error occurred. Details")
```

Outer try Block: This block attempts to open the file 'file.txt' in read mode ('r'). If the file does not exist, Python raises a FileNotFoundError, which is caught by the except FileNotFoundError block.

Inner try Block: Inside the outer try block, there's an inner try block where actual file operations (file.read()) are performed. This inner block is nested within the outer try block to ensure that if the file opening fails (due to FileNotFoundError), the file operations are not attempted.

except Blocks:

- FileNotFoundError: Handles the case where 'file.txt' does not exist or cannot be found.
- IOError: Catches any Input/Output related errors that may occur during file operations (though FileNotFoundError is a more specific case).
- Exception: Catches any unexpected errors that are not specifically handled by previous except blocks.

finally Block: Ensures that the file is closed regardless of whether an exception was raised during file operations. Closing the file in the finally block is essential for releasing system resources and maintaining proper file handling practices.

Understand how to write data to a file

Before you can write data to a file, you need to open the file in write mode using the open() function. There are two main modes for writing:

1. **"w" (Write Mode):** This mode opens a file for writing. If the file doesn't exist, it creates a new file. If the file already exists, it truncates the file and overwrites its contents.

we already have a file named file.txt, opening it in write mode ("w") will remove all the existing content from the file and overwrite it.

```
file = open("file.txt", "w")
content = file.write("Hello, World!")
print(content)
file.close()  # Close the file after writing
```

OR

```
With open("file.txt", "w") as file:
     content = file.write("Hello, World!")
     print(content)
     # No need to close the file after writing
```

The output will be:

```
13
```

The reason you're seeing 13 printed is because the write() method returns the number of characters written to the file. In this case, the string "Hello, World!" has 13 characters, so file.write("Hello, World!") returns 13.

To see the content of the file, you need to either read the file or check the file directly.

Here's how you can read the file after writing to it:

```
file = open("file.txt", "w")
file.write("Hello, World!")
file.close()  # Close the file after writing

# Reopen the file in read mode
file = open("file.txt", "r")
content = file.read()
print(content)
```

```
file.close()
```

The output will be:

```
Hello, World!
```

2. **"a" (Append Mode):** This mode opens a file for appending data at the end of the file. If the file doesn't exist, it creates a new file. when you open a file in append mode ("a"), it does not truncate (remove) the existing contents of the file.

We already have a file named file.txt. Now, let's open the file in append mode and append a new line:

```
file = open("file.txt", "a")
file.write("This is the first line.\n")
file.write("This is the second line.\n")
file.close()
```

```
# Reopen the file in read mode
file = open("file.txt", "r")
content = file.read()
print(content)
file.close()
```

The output will be:

```
Hello, World!This is the first line.
This is the second line.
```

Understand how to create a new file

We can create new files using both write ('w') and append ('a') modes. If the file doesn't exist.

Using 'x' Mode (Exclusive Creation Mode): Opens a file for exclusive creation This means that the file will be created only if it does not already exist. If the file already exists, the operation fails and raises a FileExistsError. If the file does not exist, it creates a new file and opens it for writing.

```
f = open("myfile.txt", "x")  # Create a file called "myfile.txt"
```

Result: A new empty file is created in the specified location.

If you reopen this file,

```
try:
    open('myfile.txt', 'x')
    file.write("This is a new file created using 'x' mode.\n")
    file.write("Writing to a file using 'x' mode fails if the file
already exists.\n")
except FileExistsError:
    print("Error: The file 'new_file_x.txt' already exists.")
except IOError:
    print("IOError occurred")
except Exception:
    print("An unexpected error occurred")
```

The output will be:

```
Error: The file 'myfile.txt' already exists.
```

Understand how to delete a file

Python provides the os module, which gives you access to various operating system functionalities, including the ability to delete files.

To delete a file using Python, you can use the os.remove() function.

Syntax:

```
import os
os.remove(file_path)
```

You need to provide the path to the file you want to delete.

If the file is successfully deleted, the function doesn't return any value. However, if the file doesn't exist or you don't have the necessary permissions to delete the file, it will raise an exception.

```python
import os

# Specify the file path
file_path = "path/to/file.txt"

try:
    # Delete the file
    os.remove(file_path)
    print(f"File '{file_path}' deleted successfully.")
except FileNotFoundError:
    print(f"File '{file_path}' not found.")
except PermissionError:
    print(f"You don't have permission to delete '{file_path}'.")
except Exception:
    print("An error occurred")
```

In this example, we first specify the file path. Then, we use a try-except block to handle any exceptions that may occur during the file deletion process. If the file is deleted successfully, a success message is printed. If the file doesn't exist, a FileNotFoundError is raised and handled. If you don't have the necessary permissions to delete the file, a PermissionError is raised and handled. Any other exceptions are caught by the general Exception clause.

Note: Once a file is deleted using os.remove(), it is permanently removed from the file system, and its contents cannot be recovered unless you have a backup.

Delete Folder: To delete a folder (directory) in Python, we can use the os module, specifically the os.rmdir() function. If you attempt to remove a folder that is not empty, Python will raise an OSError with the message indicating that the folder is not empty.

```python
import os
# Specify the path to the directory
directory_path = "/path/to/folder"
try:
    os.rmdir(directory_path)
    print(f"Directory '{directory_path}' has been removed
successfully.")
except FileNotFoundError:
    print(f"The directory '{directory_path}' does not exist.")
except OSError:
    print("Error")
```

In this example, we first import the os module. Then, we specify the path to the empty directory we want to remove. We use a try-except block to handle any exceptions that may occur during the directory removal process. If the directory is successfully removed, a success message is printed. If the directory doesn't exist, a FileNotFoundError is raised and handled. Any other exceptions related to operating system errors are caught by the OSError clause.

Understand the purpose and use of libraries in Python

A library is essentially a collection of modules that are bundled together.

Imagine you are a carpenter working on various furniture projects. Instead of crafting your own tools every time you start a new project, you have a toolbox filled with a variety of pre-made tools. Each tool in the toolbox serves a specific purpose, such as a hammer for driving nails, a saw for cutting wood, and a measuring tape for precision measurements.

- Toolbox: The toolbox represents the Python library. It is a collection of various tools (modules) that you can use repeatedly in different projects.
- Tools: The tools inside the toolbox are like the individual modules within a library. Each tool/module is designed to perform a specific task.
- Reuse: Just as you can use the same hammer in multiple furniture projects, you can use the same module in different Python programs.
- Efficiency: Having a toolbox with ready-to-use tools makes your work as a carpenter easier and faster. Similarly, having a library with pre-written code makes programming easier and more
- Specialization: Different tools are specialized for different tasks (cutting, measuring, hammering). Similarly, different modules are specialized for different functionalities (data manipulation, web requests, plotting).

Example Scenario
Imagine you need to build a chair. Instead of crafting a hammer from scratch each time you build a chair, you simply use the hammer from your toolbox. This saves you time and ensures you are using a reliable tool that works well. Similarly, if you need to manipulate some data in Python, instead of writing the entire data manipulation code yourself, you can use the Pandas library, which provides pre-built functions for data manipulation.

For data analytics in Python, there are several key libraries that are widely used due to their powerful features and ease of use.

1. NumPy
2. Pandas
3. Matplotlib

4. Seaborn

Understand the Importance of Database Connectivity

1. Why Database Connectivity is Essential

Introduction: In the modern world, almost every application needs to interact with a database. Databases store and manage data, which is crucial for applications ranging from simple data tracking systems to complex enterprise solutions. Let's delve into why database connectivity is essential.

Real-World Example: Imagine an online bookstore. To manage book inventory, customer information, orders, and reviews, the bookstore needs a way to store and retrieve this data efficiently. Here, a database plays a central role in managing this information, and database connectivity ensures that the application can interact with the database.

Importance of Database Connectivity:

1. **Data Storage and Management:**
   - **Persistent Storage:** Databases provide a structured way to store data persistently. Unlike in-memory data structures, data in databases remains intact across application restarts and system reboots.
   - **Organized Data:** Databases use tables and schemas to organize data efficiently. This structured organization helps in managing large volumes of data.
2. **Data Retrieval and Manipulation:**
   - **Efficient Retrieval:** Databases use indexing and querying techniques to retrieve data efficiently. This is crucial for applications that need to access and display data quickly, such as search engines or online retail platforms.
   - **Data Manipulation:** Through SQL (Structured Query Language), databases allow for complex data manipulations like updating records, deleting entries, and performing calculations.
3. **Concurrent Access:**
   - **Multi-User Access:** Databases support multiple users accessing and manipulating data simultaneously. This is essential for collaborative applications where data needs to be shared and updated by various users.
4. **Security and Integrity:**
   - **Access Control:** Databases offer mechanisms to control access to data. User permissions and roles help in ensuring that only authorized users can perform specific operations.
   - **Data Integrity:** Databases enforce rules and constraints to maintain data integrity, ensuring that the data remains accurate and consistent.

5. **Scalability:**
    - **Handling Growth:** As applications grow, databases can scale to handle increased data volume and user load. This scalability ensures that applications remain performant even as the demand increases.

Example Application: Consider a banking application. It needs to securely store customer details, transaction histories, and account balances. Database connectivity ensures that the application can safely interact with the bank's database, performing operations like account updates, transaction records, and balance checks.

Summary:

- **Data Storage:** Databases offer a structured, persistent method to store and manage data.
- **Efficient Retrieval:** Databases enable fast data retrieval and manipulation through querying.
- **Concurrent Access:** They support multiple users and concurrent data access.
- **Security:** Databases provide access control and maintain data integrity.
- **Scalability:** They handle increasing data and user load as applications grow.

Understanding the importance of database connectivity is fundamental to building robust and scalable applications. By leveraging databases, developers can ensure that their applications efficiently manage data and meet user needs effectively.

Key Takeaways:

1. Database connectivity is crucial for data persistence, organization, and efficient access.
2. It supports concurrent data access and manipulation by multiple users.
3. It ensures data security and integrity through access controls and constraints.
4. Databases can scale to handle increasing demands of data and user interactions.

Use Python Libraries for Database Connectivity

1. Python Libraries for Database Connectivity

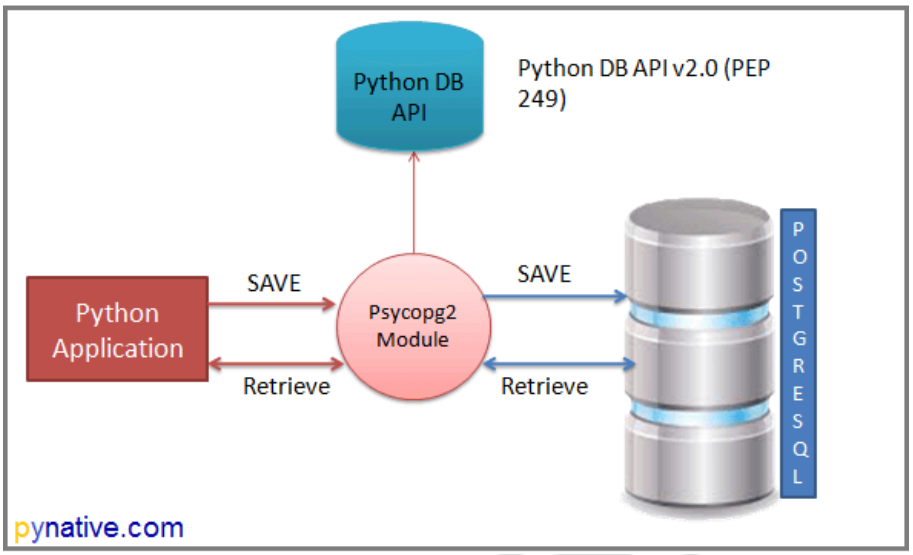
Introduction: Python offers several libraries to connect and interact with databases. These libraries abstract the complexities of database operations and provide a straightforward way to perform CRUD (Create, Read, Update, Delete) operations. Two prominent libraries for database connectivity in Python are psycopg2 and SQLAlchemy.

1.1 psycopg2

Overview: psycopg2 is a popular library used to connect to PostgreSQL databases. It is a low-level library that provides a way to interact directly with the database using SQL queries.

Key Features:

- **Direct SQL Execution:** Allows execution of raw SQL queries directly on the PostgreSQL database.
- **Transaction Management:** Supports transaction management, which ensures that database operations are completed successfully before committing changes.
- **Data Retrieval:** Facilitates retrieval of query results in various formats, such as tuples or dictionaries.



Python PostgreSQL connection to access database

Installation:

```
!pip install psycopg2
```

Example Code:

```
import psycopg2

# Connect to your PostgreSQL database
conn = psycopg2.connect(
    dbname="your_db_name",
    user="your_username",
    password="your_password",
    host="localhost",
```

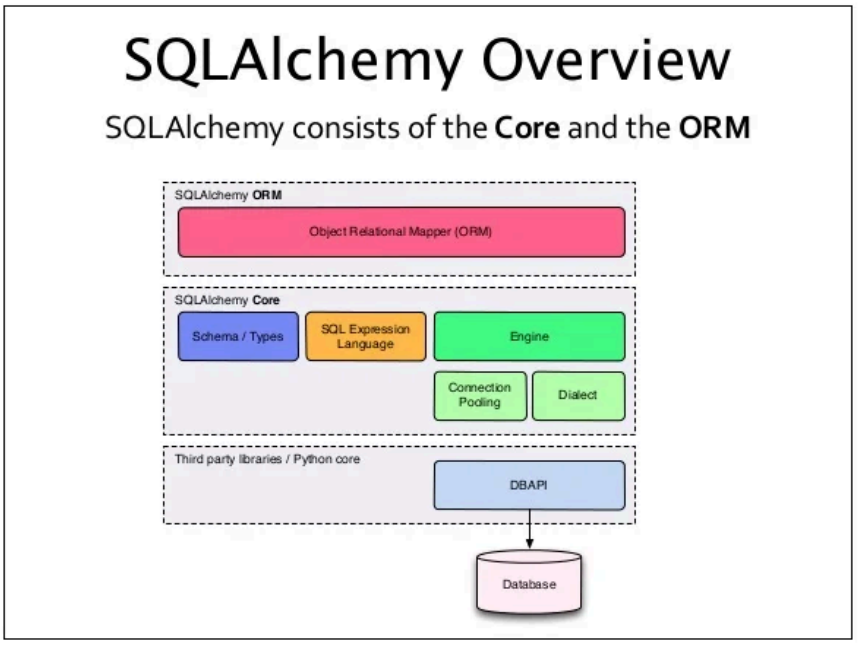
```
    port="5432"
)

# Create a cursor object
cur = conn.cursor()

# Execute a query
cur.execute("SELECT * FROM your_table_name")

# Fetch and print the results
rows = cur.fetchall()
for row in rows:
    print(row)

# Close the cursor and connection
cur.close()
conn.close()
```

Explanation:

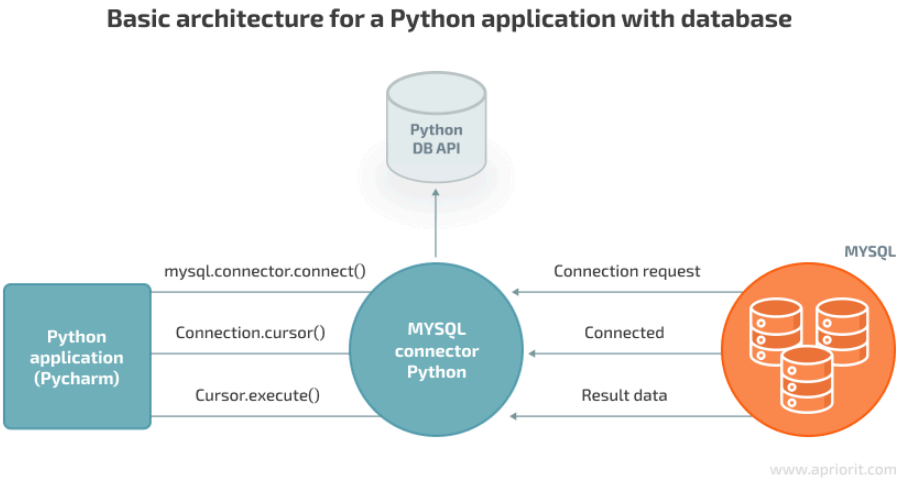
- **Connection:** Establish a connection to the PostgreSQL database with connection parameters such as database name, user, password, host, and port.
- **Cursor:** Create a cursor object to execute SQL commands.
- **Query Execution:** Use the cursor to execute SQL queries.
- **Fetching Data:** Retrieve data from the executed query using fetchall() and print the results.
- **Closing:** Ensure to close the cursor and connection to free resources.

1.2 Using SQLAlchemy for Database Operations

Overview: SQLAlchemy provides a high-level interface for database operations through its ORM and SQL toolkit. It abstracts the complexities of direct SQL execution and supports multiple databases.



SQLAlchemy Architecture



Installation:

```
!pip install sqlalchemy
```

Example Code:

```
from sqlalchemy import create_engine, Column, Integer, String
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker
```

```python
# Define the database URL
DATABASE_URL =
"postgresql://username:password@localhost:5432/your_db_name"

# Create an engine and a base class
engine = create_engine(DATABASE_URL)
Base = declarative_base()

# Define a model class
class User(Base):
    __tablename__ = 'users'
    id = Column(Integer, primary_key=True)
    name = Column(String)

# Create a session
Session = sessionmaker(bind=engine)
session = Session()

# Create the table
Base.metadata.create_all(engine)

# Add a new user
new_user = User(name="John Doe")
session.add(new_user)
session.commit()

# Query the users
users = session.query(User).all()
for user in users:
    print(user.name)

# Close the session
session.close()
```

Explanation:

- **Engine Creation:** Create an engine instance using the database URL to connect to the PostgreSQL database.
- **Model Definition:** Define a model class (User) to represent a table in the database.
- **Session Management:** Create a session to interact with the database.
- **Table Creation:** Use Base.metadata.create_all() to create tables based on the model classes.

- **Data Operations:** Add and commit new records to the database, and query existing records.
- **Session Closure:** Close the session to ensure proper resource management.

Summary:

- **psycopg2:** A library for direct SQL execution in PostgreSQL, useful for low-level database interactions.
- **SQLAlchemy:** Provides ORM and SQL toolkit capabilities, supporting multiple databases and simplifying interactions through Python classes.
- **Installation and Usage:** Both libraries are easy to install and use, each catering to different needs in database operations.

Discuss Database Connection Parameters

1. Database Connection Parameters

Introduction: To connect to a database, specific parameters are needed. These parameters provide the database system with the necessary information to locate and authenticate the user for accessing the database. Proper configuration of these parameters ensures successful communication between your application and the database server.

1.1 Essential Connection Parameters

Overview: Database connection parameters are essential for setting up a connection to a database. They include:

1. **Database Name (dbname):** Specifies the name of the database you want to access. Each database system can have multiple databases, and you need to specify which one you're targeting.
2. **User (user):** The username used to log into the database. This user must have appropriate permissions to perform operations on the database.
3. **Password (password):** The password associated with the username. This helps ensure that only authorized users can access the database.
4. **Host (host):** The address of the database server. For local development, this is often localhost. For remote servers, it could be an IP address or domain name.
5. **Port (port):** The port number on which the database server is listening for connections. Different databases use different default ports. For instance, PostgreSQL uses 5432, and MySQL uses 3306.

Example Configuration for PostgreSQL using psycopg2:

```
import psycopg2

# Define database connection parameters
dbname = "my_database"          # Name of your database
user = "my_user"                # Your username
password = "my_password"        # Your password
host = "localhost"              # Database server address
port = "5432"                   # Port number for PostgreSQL

# Establish connection to the PostgreSQL database
conn = psycopg2.connect(
    dbname=dbname,
    user=user,
    password=password,
    host=host,
    port=port
)

# Verify the connection
print("Connection successful!")
```

Explanation:

- **dbname:** The database name you want to connect to.
- **user:** The username for authentication.
- **password:** The password for the specified username.
- **host:** Address of the database server (e.g., localhost for local connections).
- **port:** The port number where the database service is running.

1.2 Configuring Connection Settings for Different Databases

Overview: Different databases use slightly different connection parameters. Here's a brief overview of how to configure connections for a few common databases:

1. **PostgreSQL:**
    - **Connection String:** postgresql://username:password@host:port/dbname
    - **Default Port:** 5432
2. Example:

```
import psycopg2

conn = psycopg2.connect(
    dbname="my_database",
```

```
    user="my_user",
    password="my_password",
    host="localhost",
    port="5432"
)
```

2. MySQL:

- **Connection String:** mysql+pymysql://username:password@host:port/dbname
- **Default Port:** 3306

Example using SQLAlchemy:

```
from sqlalchemy import create_engine

DATABASE_URL =
"mysql+pymysql://my_user:my_password@localhost:3306/my_database"
engine = create_engine(DATABASE_URL)

with engine.connect() as connection:
    print("Connection successful!")
```

3. SQLite:

- **Connection String:** sqlite:///path_to_database.db
- **Port:** Not required, as SQLite is a file-based database.

Example:

```
from sqlalchemy import create_engine

DATABASE_URL = "sqlite:///my_database.db"
engine = create_engine(DATABASE_URL)

with engine.connect() as connection:
    print("Connection successful!")
```

Explanation:

- **PostgreSQL:** Uses psycopg2 and requires the database name, user, password, host, and port.
- **MySQL:** Uses SQLAlchemy with pymysql, and the connection string includes database credentials and server details.
- **SQLite:** Uses a file path for the database, and no port is needed.

Summary:

- **Essential Parameters:** Key parameters for connecting to a database include the database name, user credentials, host address, and port number.
- **Database Variations:** Different databases have specific connection string formats and default ports.
- **Practical Usage:** Utilize provided examples to configure connections for PostgreSQL, MySQL, and SQLite in Python.

Key Takeaways:

1. **Connection Parameters:** Critical for establishing database connections; include database name, user, password, host, and port.
2. **Database-Specific Details:** Connection configurations differ based on the database type, requiring different connection strings and port numbers.
3. **Library Examples:** Use psycopg2 for PostgreSQL, SQLAlchemy for MySQL and SQLite, and understand the specific requirements for each.

Executing SQL Queries

1. Creating Databases and Tables Using Python

Introduction: Creating and managing databases and tables is fundamental to interacting with relational databases. Python libraries such as psycopg2 and SQLAlchemy enable you to execute SQL commands for database and table creation.

1.1 Creating a Database

To create a database, you typically need to connect to a database server and execute a SQL command. Note that in many systems, you may need to connect to a default database to create new databases.

Example using psycopg2 for PostgreSQL:

```
import psycopg2

# Connect to the default database to execute SQL commands
conn = psycopg2.connect(
    dbname="postgres",
    user="my_user",
    password="my_password",
    host="localhost",
    port="5432"
```

```
)
conn.autocommit = True  # Enable autocommit to execute database
creation command

# Create a cursor object
cur = conn.cursor()

# Create a new database
cur.execute("CREATE DATABASE my_new_database;")

# Close the cursor and connection
cur.close()
conn.close()

print("Database created successfully!")
```

Explanation:

- **conn.autocommit = True:** Required to run database creation commands, as they can't be executed inside transactions.
- **cur.execute("CREATE DATABASE my_new_database;"):** SQL command to create a new database.

1.2 Creating a Table

Once the database is created, you can connect to it and create tables.

Example using psycopg2 for PostgreSQL:

```
import psycopg2

# Connect to the newly created database
conn = psycopg2.connect(
    dbname="my_new_database",
    user="my_user",
    password="my_password",
    host="localhost",
    port="5432"
)

# Create a cursor object
cur = conn.cursor()
```

```
# Create a new table
cur.execute("""
    CREATE TABLE employees (
        id SERIAL PRIMARY KEY,
        name VARCHAR(100),
        position VARCHAR(50),
        salary DECIMAL
    );
""")

# Commit changes
conn.commit()

# Close the cursor and connection
cur.close()
conn.close()

print("Table created successfully!")
```

Explanation:

- **CREATE TABLE employees:** SQL command to define a new table with columns for employee information.

---

2. Retrieving Data from a Database and Managing Cursors

Introduction: Retrieving data involves executing SQL queries and fetching results using cursors. Cursors help manage the context of a SQL query and allow you to iterate over results.

2.1 Retrieving Data

Example using psycopg2 for PostgreSQL:

```
import psycopg2

# Connect to the database
conn = psycopg2.connect(
    dbname="my_new_database",
    user="my_user",
    password="my_password",
    host="localhost",
```

```
    port="5432"
)

# Create a cursor object
cur = conn.cursor()

# Execute a query to retrieve data
cur.execute("SELECT * FROM employees;")

# Fetch all rows from the result
rows = cur.fetchall()

# Print the results
for row in rows:
    print(row)

# Close the cursor and connection
cur.close()
conn.close()
```

Explanation:

- **cur.execute("SELECT * FROM employees;")**: SQL command to select all records from the employees table.
- **cur.fetchall()**: Fetches all rows from the result of the query.

2.2 Managing Cursors

Cursors help control the flow of SQL query execution and result retrieval.

- **cur.fetchone()**: Fetches a single row from the result.
- **cur.fetchmany(size)**: Fetches a specified number of rows from the result.

Example:

```
# Fetch one row at a time
cur.execute("SELECT * FROM employees;")
row = cur.fetchone()
while row is not None:
    print(row)
    row = cur.fetchone()
```

3. Common SQL Syntax in Python

Introduction: Understanding SQL syntax for various operations is crucial for effective database management. Common operations include filtering, updating, and deleting data.

3.1 SQL Syntax Examples

WHERE: Filters records based on conditions.

```
cur.execute("SELECT * FROM employees WHERE salary > 50000;")
```

UPDATE: Modifies existing records.

```
cur.execute("UPDATE employees SET salary = salary * 1.1 WHERE
position = 'Developer';")
```

DROP: Deletes tables or databases.

```
cur.execute("DROP TABLE employees;")
```

ORDER BY: Sorts records.

```
cur.execute("SELECT * FROM employees ORDER BY salary DESC;")
```

DELETE: Removes records.

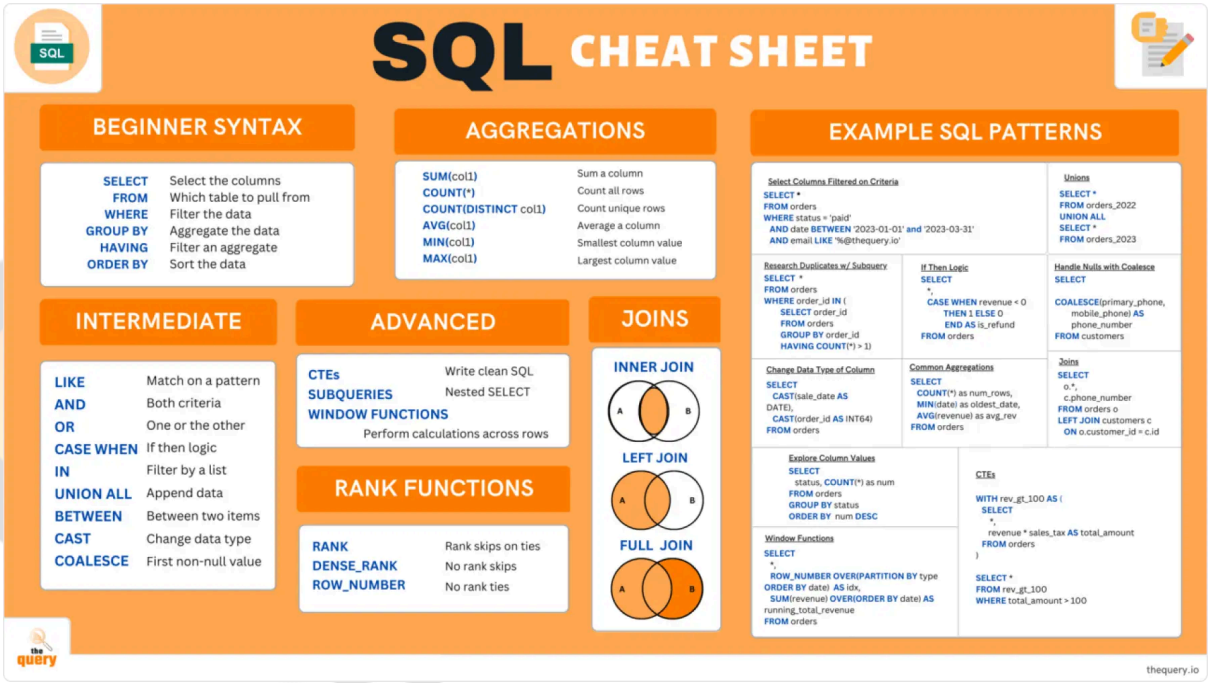
```
cur.execute("DELETE FROM employees WHERE id = 1;")
```

LIMIT: Limits the number of records returned.

```
cur.execute("SELECT * FROM employees LIMIT 5;")
```

Explanation:

- **WHERE:** Used to filter results based on specified conditions.
- **UPDATE:** Used to modify existing records.
- **DROP:** Used to delete database objects.
- **ORDER BY:** Used to sort results.
- **DELETE:** Used to remove specific records.
- **LIMIT:** Used to limit the number of results returned.



4. Practical Implementation of Database Connectivity and SQL Queries

Introduction: Practical implementation involves connecting to a database, executing SQL queries, and handling results in real scenarios.

Example:

1. **Create a Database and Table:**

```
import psycopg2

conn = psycopg2.connect(
    dbname="my_new_database",
    user="my_user",
    password="my_password",
    host="localhost",
    port="5432"
)
cur = conn.cursor()

cur.execute("""
    CREATE TABLE products (
        id SERIAL PRIMARY KEY,
        name VARCHAR(100),
        price DECIMAL
    );
```

```
""")
conn.commit()

cur.close()
conn.close()
```

Insert and Retrieve Data:

```
import psycopg2

conn = psycopg2.connect(
    dbname="my_new_database",
    user="my_user",
    password="my_password",
    host="localhost",
    port="5432"
)
cur = conn.cursor()

# Insert data
cur.execute("""
    INSERT INTO products (name, price)
    VALUES ('Laptop', 999.99), ('Smartphone', 499.99);
""")
conn.commit()

# Retrieve data
cur.execute("SELECT * FROM products;")
rows = cur.fetchall()
for row in rows:
    print(row)

cur.close()
conn.close()
```

Explanation:

- **Creating and Inserting Data:** Demonstrates creating tables and inserting records into them.
- **Retrieving Data:** Shows how to fetch and display data from the database.

Summary:

Creating Databases and Tables:

- **Database Creation:** Establish a new database using SQL commands in Python. This involves defining the database structure to store and organize data.
- **Table Definition:** Create tables within the database to hold specific types of information, such as user details or product records. This is done by specifying table names, columns, and data types.

Retrieving Data and Managing Cursors:

- **Data Retrieval:** Use SQL SELECT queries to extract data from tables. This includes filtering and selecting specific records based on conditions.
- **Cursor Management:** Employ cursors to navigate through query results and fetch data efficiently. Cursors help manage the data retrieval process and ensure accurate handling of results.

Executing Common SQL Commands:

- **Filtering Data (WHERE):** Apply conditions in SQL queries to retrieve specific records that match certain criteria.
- **Updating Records (UPDATE):** Modify existing records in a table based on specified conditions.
- **Deleting Records (DELETE and DROP):** Remove specific records or entire tables from the database as needed.
- **Sorting Results (ORDER BY):** Arrange query results in a desired order, such as ascending or descending by a particular column.
- **Limiting Results (LIMIT):** Restrict the number of records returned by a query to manage the volume of data processed.

Practical Implementation:

- **Connecting and Querying:** Establish a connection to a database and execute SQL commands to perform various operations, including creating tables, inserting data, and querying information.